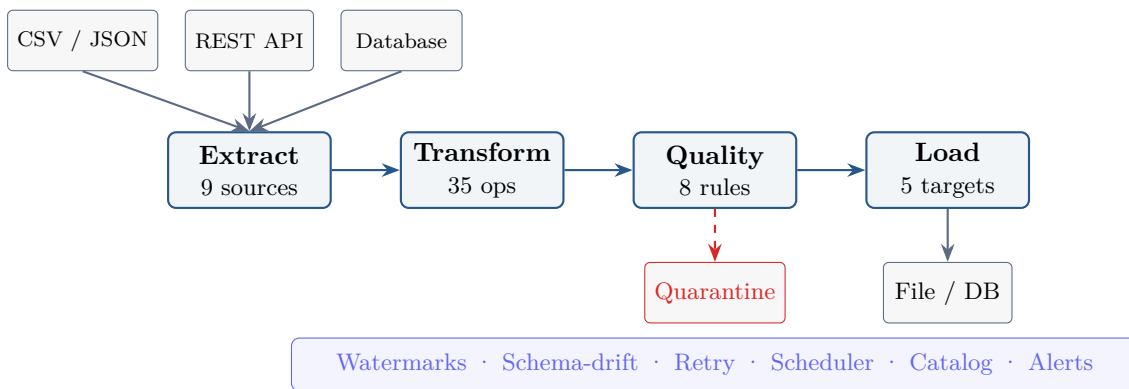


DataForge ETL

A Local-First Data Pipeline & Orchestration Engine

Architecture, Connectors, Data Quality, Scheduling, and Cataloguing



*Pipeline lifecycle and cross-cutting services (schematic, not to scale).
Every stage is declarative and driven by a single YAML definition.*

Author: Salman K. Alwajaan

Contact: salman.khalid.alwajaan@gmail.com

Platform: macOS desktop application (local-first)

Document: Revision 3.0 · 2026

Contents

1	Executive Summary	3
2	System Overview	3
2.1	Design Goals	3
2.2	Technology Stack	3
3	Architecture	4
3.1	Pipeline Lifecycle	4
4	Connectors	4
4.1	Extractors (data sources)	5
4.2	Transformers	5
4.3	Loaders (destinations)	5
5	Data Quality Engine	5
6	Incremental Loading & Schema Drift	6
7	Orchestration	6
7.1	Scheduler	6
7.2	Pipeline Chaining	6
7.3	Backfills	6
8	Data Catalog	7
9	Web Dashboard	7
9.1	Extract Data workspace	7
9.2	Pipeline Editor	7
10	Command-Line Interface	8
11	Example Pipeline	8
12	Validation & Results	10
13	Deployment	10
Appendix A	Full Connector Inventory	10

1 Executive Summary

DataForge ETL is a local-first data pipeline and orchestration engine. It ingests data from nine kinds of source, applies a library of thirty-five composable transformations, validates the result against a declarative data-quality contract, and writes to any of five destinations—all described by a single, human-readable YAML file. The engine runs entirely on the user's machine: there is no cloud dependency, no external service, and no data leaves the host.

The system ships in three interchangeable forms that share one engine:

- a **native desktop application** (a windowed macOS `.app`);
- a **web dashboard** served locally at `127.0.0.1:8765`; and
- a **command-line interface** for scripting and automation.

At a glance

Sources	CSV, JSON, Excel, Parquet, SQL database, REST API, synthetic generator, inline, multi-file glob
Transforms	35 operations incl. SQL-on-DataFrame, window analytics, PII masking, binning
Quality	8 rule types incl. schema contracts and z-score anomaly detection
Loaders	CSV, JSON, Excel, Parquet, SQL database (append / replace / upsert)
Orchestration	Cron scheduler, pipeline chaining, variable backfills, retries
Governance	Incremental watermarks, schema-drift detection, data catalog, run history

2 System Overview

2.1 Design Goals

DataForge ETL was built around four principles:

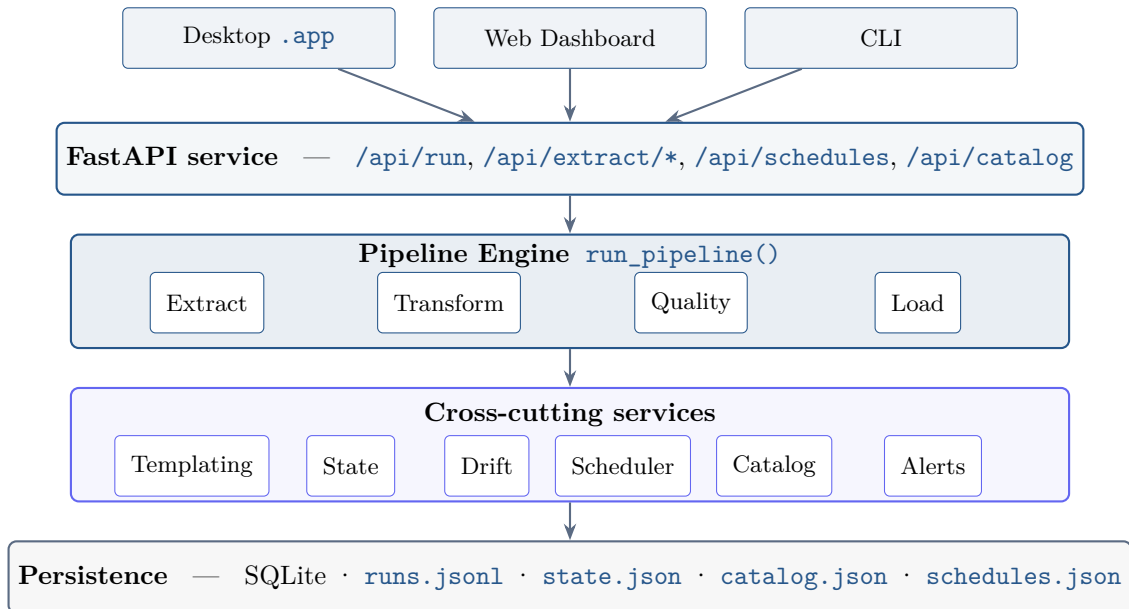
1. **Declarative.** A pipeline is data, not code. The complete behaviour of an ingestion job is captured in one YAML document that can be version-controlled, diffed, and reviewed.
2. **Local-first.** The engine, the dashboard, the scheduler, and all state live on the user's machine. Run history, watermarks, schedules, and the data catalog are persisted under `~/.etl-engine/`.
3. **Composable.** Extractors, transformers, loaders, and quality rules are registered in a central registry; new capabilities are added by dropping a class into the registry, with no change to the runner.
4. **Observable.** Every run records stage timings, throughput, rejected-row counts, and schema drift, surfaced in both the dashboard and the CLI.

2.2 Technology Stack

Layer	Implementation
Compute engine	Python 3, <code>pandas</code> DataFrames
Web service	<code>FastAPI</code> + <code>uvicorn</code> (ASGI)
Desktop shell	<code>pywebview</code> native window wrapping the local server
Persistence	SQLite (via <code>SQLAlchemy</code>) and JSON state files
CLI	<code>Typer</code> + <code>rich</code> terminal rendering
Front end	Single-file vanilla-JS SPA, inline SVG iconography

3 Architecture

The engine is organised in layers. Requests from the desktop shell, the browser, or the CLI all converge on one function, `run_pipeline()`, which orchestrates the extract–transform–quality–load lifecycle and the cross-cutting services around it.



3.1 Pipeline Lifecycle

For every run the engine executes the following ordered steps. Any step may emit warnings without failing the run; only quality failures configured as fatal, or an unrecoverable extraction error, abort the pipeline.

1. **Render.** Resolve `{{ variable }}` placeholders from the pipeline’s own `variables` block, runtime overrides, and environment variables.
2. **Extract.** Pull the source, retrying with exponential back-off if configured.
3. **Drift check.** Compare the raw schema against the previous run’s stored schema.
4. **Incremental filter.** Keep only rows newer than the stored watermark.
5. **Transform.** Apply each transformation in declared order.
6. **Quality.** Evaluate each rule; fail the run, or quarantine offending rows.
7. **Load.** Write to the destination (append, replace, or upsert).
8. **Persist & notify.** Update watermark, schema, catalog, run history; fire webhook alerts; trigger any chained pipelines.

4 Connectors

4.1 Extractors (data sources)

Type	Description
<code>csv</code>	Delimited files; optional chunked reads for very large files
<code>json</code>	JSON and newline-delimited JSON, with nested record paths
<code>excel</code>	<code>.xlsx/.xls</code> workbooks, sheet selectable
<code>parquet</code>	Columnar Parquet, column pruning
<code>database</code>	Any SQL database via SQLAlchemy (table or custom query)
<code>api</code>	REST endpoints with auth, pagination, and nested extraction
<code>generate</code>	Synthetic data generator (sequences, names, choices, dates...)
<code>inline</code>	Records embedded directly in the pipeline YAML
<code>glob</code>	Union of many files matching a pattern, with source tagging

4.2 Transformers

Thirty-five transformations are grouped by purpose. Each is a small, declarative step.

Group	Operations
Filtering	<code>filter_rows</code> , <code>select_columns</code> , <code>drop_columns</code> , <code>deduplicate</code> , <code>drop_nulls</code> , <code>sample</code>
Mapping	<code>cast_types</code> , <code>add_column</code> , <code>fill_nulls</code> , <code>normalise_text</code> , <code>map_values</code> , <code>sort</code> , <code>string_split</code>
Aggregation	<code>group_by</code> , <code>pivot</code> , <code>melt</code> , <code>window</code>
Enrichment	<code>join</code> , <code>infer_schema</code> , <code>add_timestamp</code> , <code>explode</code>
Derivation	<code>rename</code> , <code>regex_replace</code> , <code>date_extract</code> , <code>concat</code> , <code>calculate</code> , <code>mask_pii</code> , <code>bin</code> , <code>round_values</code> , <code>clip</code>
Advanced	<code>sql</code> , <code>rank</code> , <code>cumulative</code> , <code>lag</code> , <code>flatten_json</code>

The `sql` transformer is notable: it registers the working DataFrame as an in-memory table named `df` and lets the user run arbitrary SQL against it, bridging declarative pipelines and full relational expressiveness.

4.3 Loaders (destinations)

Type	Modes
<code>csv</code> , <code>json</code> , <code>excel</code> , <code>parquet</code>	write, append
<code>database</code>	append, replace, fail, upsert (merge on key columns)

5 Data Quality Engine

Quality rules run after transformation. Each rule reports a pass/fail verdict, a count of offending rows, and a human-readable message. A failing rule can either abort the pipeline (`fail_pipeline`) or *quarantine* the offending rows to a side file (`reject: true`) so that clean rows still load.

Rule	Checks that...
<code>not_null</code>	specified columns contain no nulls
<code>unique</code>	specified columns form a unique key
<code>range</code>	numeric values fall within [min, max]
<code>regex</code>	string values match a pattern
<code>allowed</code>	values belong to an allowed set
<code>expression</code>	every row satisfies a boolean expression
<code>row_count</code>	the dataset size is within bounds
<code>freshness</code>	timestamps are newer than a max age
<code>schema</code>	columns and dtypes match a declared contract
<code>zscore</code>	the share of statistical outliers stays under a threshold

Example: schema contract + anomaly guard

A `schema` rule fails the run if a column disappears or changes type—catching upstream breakage before bad data propagates. A `zscore` rule flags numeric outliers ($|z| > 4$) and fails only if they exceed an allowed percentage, catching data-entry errors and sensor spikes without rejecting legitimate variance.

6 Incremental Loading & Schema Drift

For large or growing sources, DataForge ETL supports **watermark-based incremental extraction**. The engine remembers the maximum value of a chosen column (for example an `id` or `updated_at`) from the previous successful run and, on the next run, keeps only rows beyond that watermark. State is stored in `~/.etl-engine/state.json` and can be inspected or reset per pipeline.

Alongside the watermark, the engine stores the source schema. On each run it compares the current schema to the stored one and reports **drift**—added columns, removed columns, or changed types. Drift can be advisory (a warning plus a dashboard alert) or fatal (`fail_on_drift`). A dedicated `on_drift` webhook can notify external systems.

7 Orchestration

7.1 Scheduler

A built-in cron scheduler runs pipelines on a timetable. Schedules are **persisted** to disk and reloaded when the engine restarts, so automation survives a reboot. Each schedule references a saved pipeline file and may carry its own variable overrides.

7.2 Pipeline Chaining

A pipeline may declare a `chain` of downstream pipelines to run automatically on success. This expresses dependencies—for example, a raw-ingestion job that triggers a roll-up aggregation once its data lands. Chaining is depth-limited to prevent cycles.

7.3 Backfills

The `backfill` command (and `/api/backfill` endpoint) sweeps a variable across a list of values or a date range, running the pipeline once per value. This re-processes historical partitions—for

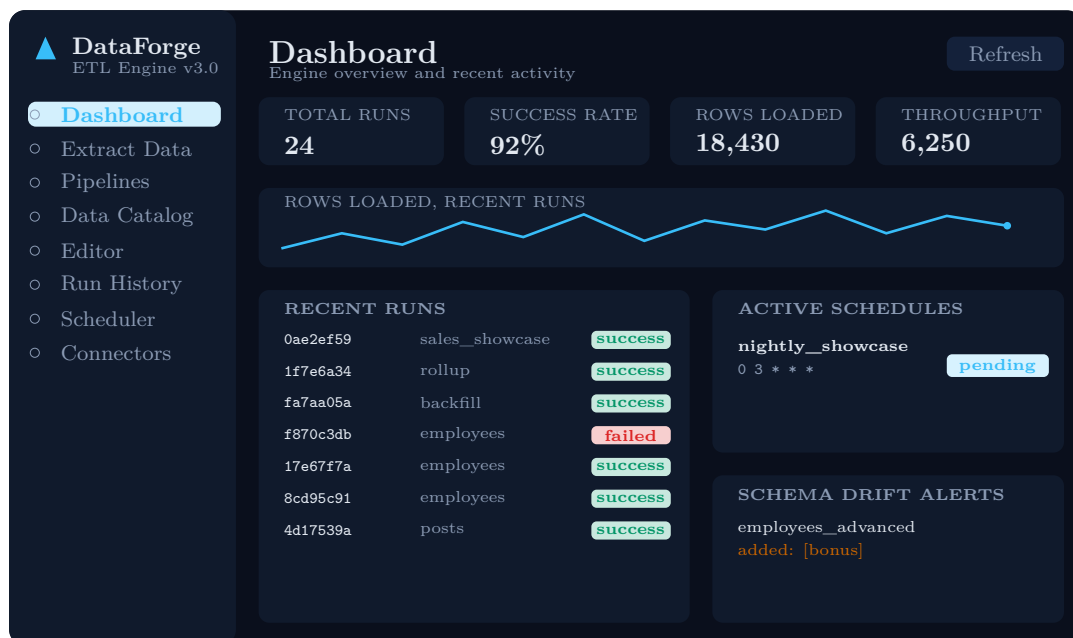
instance, regenerating one output file per day across a month. The sweep stops on the first failure to avoid compounding errors.

8 Data Catalog

Every successful load registers its destination in a **data catalog** (`catalog.json`). Each entry records the dataset location, its schema, the last row count, how many times it has been loaded, the producing pipeline, and the last-updated timestamp. The catalog gives a single, searchable inventory of every dataset the engine has ever produced—answering “what data exists, where, and who made it.”

9 Web Dashboard

The dashboard is a single-file browser application served by the local engine. It uses inline SVG iconography throughout (no emoji, no icon fonts) and offers dark and light themes. The following figure reproduces the **Dashboard** view.



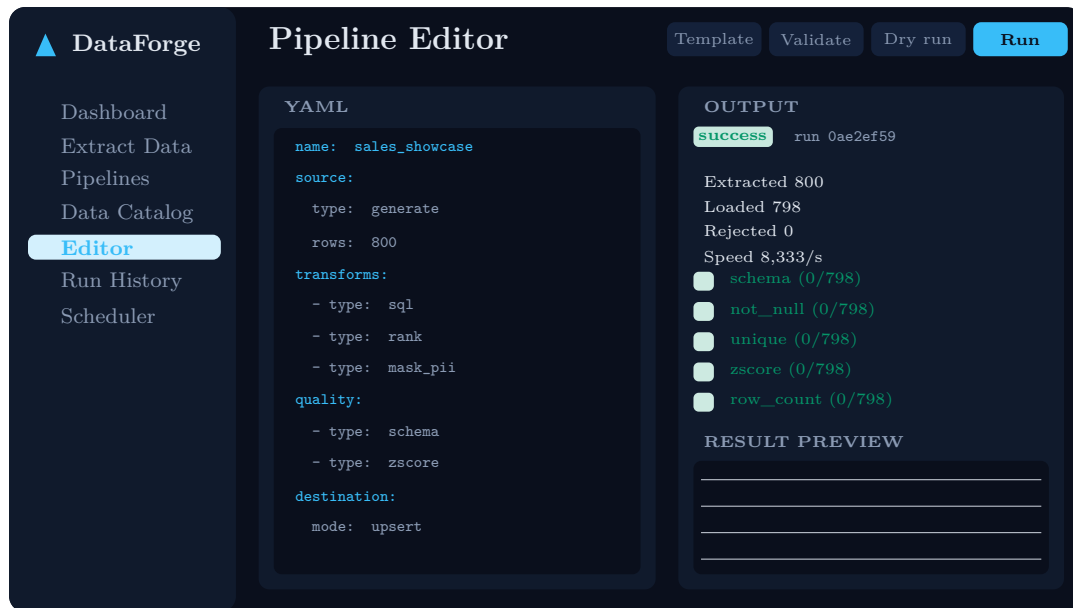
Dashboard view: summary statistics, a throughput sparkline, recent-run history with status badges, active schedules, and schema-drift alerts.

9.1 Extract Data workspace

A dedicated workspace lets an analyst point at any source, **preview** typed rows, **profile** the data (per-column nulls, distinct counts, ranges, and top values), and **export** the result to CSV, JSON, Excel, or Parquet—without writing a pipeline. This turns the tool into an ad-hoc data-exploration console as well as a batch engine.

9.2 Pipeline Editor

The editor provides a YAML workspace with a one-click template, live validation, dry-run and full-run execution, and result previews. The next figure reproduces the **Editor** view.



Editor view: YAML on the left; validation, run status, per-rule quality results, and a result preview on the right.

10 Command-Line Interface

The CLI exposes the full engine for scripting and CI use.

Command	Purpose
<code>dataforge run <file></code>	Run a pipeline; <code>-V key=value</code> overrides variables
<code>dataforge extract <src></code>	One-shot extract to any format, no pipeline
<code>dataforge profile <src></code>	Print per-column statistics for a source
<code>dataforge backfill <file></code>	Sweep a variable over values or a date range
<code>dataforge state</code>	Show incremental watermarks and stored schemas
<code>dataforge history</code>	Recent run history with throughput
<code>dataforge connectors</code>	List all extractors, transformers, loaders
<code>dataforge serve</code>	Start the web dashboard
<code>dataforge desktop</code>	Launch the native desktop window

11 Example Pipeline

The following complete definition exercises most of the engine: a synthetic source, a SQL transform, window analytics, PII masking, a schema contract, an anomaly guard, an upserting database load, and an automatic downstream roll-up via `chain`.

```
name: sales_showcase
description: Generate, SQL, rank, cumulative, schema contract, upsert, chain

variables:
  region_filter: ""

source:
  type: generate          # synthetic data for demos and tests
```



```
rows: 800
seed: 11
columns:
  order_id: {type: sequence, start: 1000}
  customer: {type: name}
  email:    {type: email, null_pct: 2}
  amount:   {type: float, min: 5, max: 1200, round: 2}
  region:   {type: choice, values: [east, west, north, south]}

transforms:
- type: sql                # full SQL over the frame (table name: df)
  query: "SELECT *, ROUND(amount*0.15,2) AS tax FROM df WHERE amount >= 10"
- type: cumulative
  column: amount
  by: [region]
  target: region_running_total
- type: rank
  column: amount
  by: [region]
  target: rank_in_region
- type: mask_pii
  columns: [email]
  method: hash

quality:
  rules:
  - type: schema           # contract: fail if shape changes
    columns: {order_id: int64, amount: float64}
  - type: unique
    columns: [order_id]
  - type: zscore           # anomaly guard
    column: amount
    threshold: 4
    max_pct: 1

destination:
  type: database
  connection_string: sqlite:///tmp/dataforge.db
  table: sales_orders
  mode: upsert
  keys: [order_id]

chain:
- chain_rollup.yaml       # runs automatically on success

alerts:
  on_failure: http://127.0.0.1:8765/api/hooks/echo
  on_drift:   http://127.0.0.1:8765/api/hooks/echo
```

12 Validation & Results

The engine was exercised end-to-end. Representative results from the showcase pipeline and its supporting tests are summarised below.

Scenario	In	Out	Outcome
Showcase: generate → SQL → window → upsert	800	798	success , 5/5 rules pass
Chained roll-up (auto-triggered)	798	4	success
Incremental second run (no new rows)	800	0	success
Incremental delta after +20 rows	220	20	success
Schema drift (new bonus column)	220	20	drift flagged
Quality quarantine (null department)	200	192	8 rows quarantined
Backfill sweep over 4 dates	200	200	4/4 success
Glob union of 4 files	200	200	4 source files tagged
Schema-contract negative test	3	—	correctly failed
Retry with back-off (missing file)	—	—	3 attempts, then alert

Across runs the engine sustained throughput on the order of several thousand rows per second on commodity hardware, with sub-second end-to-end latency for datasets of a few hundred to a few thousand rows.

13 Deployment

DataForge ETL installs as a self-contained macOS application. The engine boots a local server on `127.0.0.1:8765` and opens a native window; if the server is already running it is reused. All state is kept under `~/.etl-engine/`. The same codebase also runs headless via `dataforge serve` for server deployments, or fully scripted via the CLI in a CI pipeline.

Persisted state (survives restarts)

<code>runs.jsonl</code>	append-only run history
<code>state.json</code>	incremental watermarks + last-seen schemas
<code>catalog.json</code>	data catalog of every dataset produced
<code>schedules.json</code>	cron schedules, reloaded on boot

Appendix A Full Connector Inventory

Extractors (9): csv, json, excel, parquet, database, api, generate, inline, glob.

Loaders (5): csv, json, excel, parquet, database.

Transformers (35): filter_rows, select_columns, drop_columns, deduplicate, drop_nulls, sample, cast_types, add_column, fill_nulls, normalise_text, map_values, sort, string_split, group_by, pivot, melt, window, join, infer_schema, add_timestamp, explode, rename, regex_replace, date_extract, concat, calculate, mask_pii, bin, round_values, clip, sql, rank, cumulative, lag, flatten_json.

Quality rules (10): not_null, unique, range, regex, allowed, expression, row_count, freshness, schema, zscore.

DataForge ETL — Revision 3.0 — Prepared by Salman K. Alwajaan — 2026